



FS



← Découvrez les librairies Python pour la Data Science



6% de progression



Créez votre premier data frame avec Pandas

02:43

Préparez votre jeu de données

Comme nous avons pu le voir, les arrays NumPy sont particulièrement efficaces pour traiter des valeurs numériques. Mais les données, dans la réalité, ne sont pas composées uniquement de chiffres et de nombres.

En effet, on retrouve aussi :

- des catégories ;
- des labels ;
- des dates ;
- du texte brut.

De plus, ces données ont généralement un format prédéfini en analyse de données, où chaque ligne va correspondre à un individu (au sens statistique du terme), et chaque colonne va être une caractéristique spécifique des individus. C'est ce que l'on appelle une **variable**.

Le terme de *variable* lorsqu'on parle de données est à



sujet.

Voici quelques exemples pour illustrer cela :

- dans le milieu automobile, chaque individu sera une voiture, et on pourra avoir comme caractéristiques la puissance du moteur, les dimensions du véhicule, la marque, le modèle, la couleur, etc. ;
- dans une étude de grande distribution, chaque individu pourra être un produit sur lequel on aurait plusieurs informations (le prix, la catégorie, etc.) ;
- dans le milieu bancaire enfin, chaque individu sera une personne, sur laquelle on aurait enregistré le salaire moyen, le genre, ses mensualités de remboursement de prêt, etc. ;

Voilà par exemple à quoi pourrait ressembler ce dernier cas :

	identifiant	ville	CP	revenu	remboursement	duree	type	taux	email	nom	genre
0	0	TOULOUSE	31100	3669.0	1130.05	240	immobilier	1.168	LaurentDagenais@rhyta.com	Laurent Dagenais	M
1	1	PARIS	75009	5310.0	240.00	64	automobile	3.701	GuyMarois@fleckens.hu	Guy Marois	M
2	1	PARIS	75009	5310.0	1247.85	300	immobilier	1.173	GuyMarois@fleckens.hu	Guy Marois	M
3	2	MARSEILLE	13010	1873.0	552.54	240	immobilier	0.972	BeaufortLesage@einrot.com	Beaufort Lesage	M
4	3	MARSEILLE	13010	1684.0	586.03	180	immobilier	1.014	RussellDurand@armyspy.com	Russell Durand	M

Base de données au format Excel

En explicitant le format des données, vous avez peut-être eu l'image d'un fichier **Excel**, et c'est une bonne idée ! Excel est encore pour beaucoup d'entreprises un format très utilisé pour stocker et déplacer des données. Mais ce n'est pas le seul, les données peuvent être stockées sous bien des formats différents.

Par exemple, on retrouve régulièrement des **fichiers texte** ou des **fichiers CSV** (pour comma-separated values). Ce sont simplement des fichiers contenant l'ensemble des données brutes, séparées par un **délimiteur**.

Voici un exemple avec [un jeu de données automobiles](#), dont le délimiteur est le point-virgule :

car.csv - Bloc-notes

Fichier Edition Format Affichage Aide

Modele;CYL;PUISS;LONG;LARG;POIDS;V_MAX



```
name 100;100;20;100;10;100;100
Simca 1300;1294;68;424;168;1050;152
Citroen GS Club;1222;59;412;161;930;151
```

Exemple de fichier CSV

Cela peut également être sous la forme de fichier **JSON**. Le JavaScript Object Notation (JSON) est un format standard utilisé pour représenter des données structurées. Cela ressemble à un gros dictionnaire Python pouvant contenir lui-même d'autres dictionnaires et/ou listes. C'est une description un peu "réductrice", mais une image vaut mieux que mille mots :

indiv.json - Bloc-notes

Fichier Edition Format Affichage Aide

```
{
  "prenom": "Benjamin",
  "nom": "Marlé",
  "adresse": {
    "rue": "quai neuf trois quart",
    "ville": "Paris",
    "code postal": "75001",
    "pays": "France"
  },
  "mails": [
    "benjamin.marle@openclassrooms.fr",
    "benjamin.marle@yahoo.fr"
  ]
}
```

Exemple de fichier JSON

Nous n'aurons malheureusement pas l'occasion d'approfondir le

format JSON dans le cadre de ce cours, mais il existe [de nombreuses ressources](#) qui traitent le sujet en détail. Le JSON est un format très standard dans le monde informatique !

Nous n'avons ici vu que les formats texte, JSON et CSV, mais la liste n'est naturellement pas exhaustive. De plus, plutôt que de



succinctement ici car il fera **l'objet d'un autre cours** !

Nous cherchons donc un outil qui nous permette de représenter les données avec le format souhaité (individus/variables), de manipuler différents types de données, et de lire les données provenant de différentes sources. Cet outil n'est autre que la librairie **Pandas**, et plus particulièrement les objets **data frame**.

Le data frame est un objet Python permettant de représenter les données sous forme de tableau, où chaque colonne est explicitement nommée. Il reprend les mêmes paradigmes que l'array NumPy : chaque colonne peut naturellement être d'un type différent, mais une colonne ne peut contenir qu'un seul type ! Cette organisation simplifie l'accès aux variables, et permet de nombreuses manipulations de données plus ou moins complexes.

Mais du coup, quel est l'intérêt de la librairie NumPy si Pandas permet de faire tout cela ?

Excellente question ! Vous verrez au fur et à mesure de ce cours, et plus globalement de vos futures analyses de données, que vous serez régulièrement amené à jongler entre les deux. Il est assez courant que certaines méthodes appliquées à des data frames retournent des arrays, et qu'on ait besoin de retransformer ces arrays en data frames à posteriori. C'est pourquoi il est nécessaire de maîtriser les deux librairies, pour être parfaitement armé avant de plonger dans une analyse de données.

En parlant de *plonger*, que diriez-vous de créer votre premier data frame ?

Générez votre premier data frame

Nous allons créer notre premier data frame à partir des formats cités ci-dessus ; et comme lorsqu'on aime, on ne compte pas... je vous propose d'importer **le même** jeu de données... sous **3 formats** différents : format Excel, format CSV et format JSON.



02:49

Il est important de placer les fichiers csv dans le même dossier que le notebook jupyter pour en faciliter l'importation !

Une fois le data frame importé, commence alors le travail principal d'un analyste de données : la manipulation de données. Et avant toute manipulation, il est nécessaire de **connaître** son jeu de données.

Identifiez les caractéristiques de votre data frame

Pandas met à disposition plusieurs méthodes pour pouvoir faire cela de façon efficace.

Aperçu du data frame

Comme présenté ci-dessus, un bon réflexe à adopter après l'importation, et après toute transformation importante, est de visualiser le jeu de données, ou du moins quelques lignes, afin de vérifier que

tout s'est correctement déroulé.

Pour cela, il existe deux méthodes principales :

- la méthode `.head()` permettant de sélectionner par défaut les 5 premières lignes du data frame. Il est possible de préciser entre parenthèses le nombre de lignes à afficher ;



Les dernières lignes du data frame, il est également possible de préciser entre parenthèses le nombre de lignes à afficher.

Il n'est pas possible (par défaut) d'afficher plus de 60 lignes d'un data frame, afin de ne pas surcharger inutilement le notebook. De façon plus globale, chercher à visualiser l'ensemble d'un data frame n'est généralement pas une bonne pratique. Si cela est tout à fait envisageable avec quelques dizaines de lignes, ça devient vite impossible avec plusieurs millions de lignes ! Si vous cherchez à afficher plus de 60 lignes, vous aurez finalement comme résultat les 5 premières et dernières lignes du data frame.

Voici quelques exemples :

```
1 # chargement du fichier
2 import pandas as pd
3 data = pd.read_csv("clients.csv")
```

```
1 # afficher les 5 premières lignes
```

```
2 display(data.head())
```



```
1 # afficher les 2 dernières lignes  
2 display(data.tail(2))
```

```
1 # afficher les 5 premières et dernières lignes  
2 display(data)
```

On peut aussi accéder facilement aux caractéristiques globales d'un data frame.

Caractéristiques globales du data frame

Lorsqu'on parle de caractéristiques globales, il est question ici des attributs, des informations générales qu'on retrouve sur tous les data frames, et dont on aura besoin régulièrement.

En premier lieu viennent les dimensions d'un data frame. Combien de lignes comportent un data frame ? Et combien de colonnes ? Tout comme avec les arrays NumPy, il est possible de répondre à ces questions via l'attribut `.shape` :

```
1 data.shape
```



colonnes. On peut naturellement stocker le résultat de cet attribut dans une variable pour réutiliser ces éléments ultérieurement :

```
1 dim = data.shape
2 print(dim[0]) # 228
3 print(dim[1]) # 4
```

Au-delà des dimensions, on peut avoir envie de connaître les types de chacune de nos variables. On peut accéder à cela très simplement à partir de l'attribut `.dtypes` :

```
1 data.dtypes
```

Vous devriez avoir l'affichage suivant :

```
identifiant    int64
email          object
nom            object
genre          object
dtype: object
```

Affichage du type de chaque variable

Vous noterez que le type de l'e-mail, du nom et du genre est `objet`, alors que nous avons pourtant des chaînes de caractères. C'est une chose à connaître, mais le type `objet` de Pandas correspond en fait à une colonne de type chaîne de caractères (ou `string`).

Bien qu'avec un affichage un peu austère, cela permet d'avoir facilement le type de chaque variable de notre data frame.

Enfin, nous avons évoqué précédemment le lien qui peut exister entre

Pandas et NumPy. Je vous propose dès à présent de matérialiser ce lien en transformant notre data frame en array :

```
1 clients_array = data.values
2 display(clients_array)
```

Et tada ! Aussi simplement que cela, nous avons à présent un array

à disposition pour nos analyses.



En resume

- En analyse de données, les données sont généralement représentées et manipulées sous un format de tableau, où chaque ligne représente un individu, et chaque colonne une variable.
- Le data frame de Pandas propose une implémentation de ce format en Python.
- Les formats de stockage des données sont multiples : cela peut être via des fichiers délimités (CSV ou texte), ou encore des fichiers JSON ou Excel.
- L'objet data frame de Pandas permet de manipuler simplement et efficacement les données :
 - en important très facilement tous ces différents formats ;
 - en accédant facilement aux caractéristiques générales de notre jeu de données, comme les types de variables, le nombre de lignes, le nombre de colonnes, etc.

Maintenant que nous sommes au point sur les informations générales d'un data frame, il est à présent temps de découvrir un peu plus en détail ce que Pandas nous permet de faire en termes de manipulation de data frame.



[Avez-vous une suggestion pour nous ?](#)